CogLean: Towards a Machine-Checkable Library of Cognitive Theory

Jeff Shrager

Draft, November 2026

Draft for Comment[1]

Abstract

CogLean is a library of cognitive theory expressed in the Lean programming language and proof assistant, in which the definitions that run as models are the same objects that theorems are about. Its premise is that once we write down what we already believe about a cognitive theory with mathematical precision, we can discover things about that theory that we did not know. This paper is an invitation to cognitive scientists, and assumes no knowledge of Lean. It asks four questions a cognitive scientist might ask of such a library and answers each with one short demonstration. Does a theory entail what its authors said? Siegler and Shrager's 1984 model of strategy choice is checked: its "nonintuitive prediction" is proved, and a claim in the same chapter turns out to need a condition the chapter does not state, a condition the formalization found. Can a theory be shown unable to produce a phenomenon? Collins and Quillian's hierarchical network can never show a typicality effect, for any network, while a feature comparison model can. Do independently formalized theories compose? EPAM, the strategy-choice model and a problem solver compose into a model of three-digit addition, a pattern of confident errors is proved for the composition and for every memory satisfying three laws, a simplified ACT-R memory reaches the pattern by another route, and the two explanations differ on an experiment nobody has run, which the comparison produced rather than presupposed. Can theories be compared by what they entail rather than how they fit? A memory theory matrix classifies four formalized memory families against six laws of retrieval, every verdict a theorem. What these discoveries establish is about the theories, not directly about people; their psychological significance still turns on empirical assumptions. The goal is to grow CogLean into a Mathlib-like repository of machine-checkable cognitive theory, in which such results accumulate, and the paper closes with what each kind of cognitive scientist could contribute to it.

1 Four questions

Suppose a cognitive theory were written down precisely enough that a machine could check what follows from it. What would we want to ask? Not, in the first place, whether the theory fits a data set; we have good tools for that. The questions I have in mind are the ones that come up when theories are discussed rather than fitted. Does the theory, as written, actually entail the predictions its authors drew from it? Is there a phenomenon it *cannot* produce, whatever its

[1] This is a working draft. I welcome corrections, constructive challenges, and especially proposals for collaboration. Email me: `jshrager@gmail.com`.

parameters? When two theories explain the same phenomenon, do they explain it for the same reasons, and what observation would tell them apart? And can theories be compared by what they entail, rather than by how well each can be made to fit? Behind all of these is one idea. If we can write down what we already believe about a theory with mathematical precision, then the theory becomes something we can reason about mechanically, and we can discover consequences, limitations and unstated assumptions that we did not know were there, and perhaps would not have found otherwise. Formalization is the necessary first step; discovery is the payoff.

These questions are old, and cognitive scientists have always answered them by argument, by simulation, and sometimes by mathematics. What has been missing is a place where the answers accumulate. Allen Newell famously complained that cognitive science was collecting independent phenomena and theories ("playing twenty questions with nature") instead of assembling unified understandings of cognition [1]. Cognitive architectures such as Soar [2, 3] and ACT-R [4], and the Common Model of Cognition [5], are direct responses to Newell's complaint. Although such architectures bring together multiple mechanisms to explain multiple phenomena, they are usually rendered either as computational models (simulations), or as principles asserted about such models. Cognitive principles have been stated in formalisms that can be checked computationally [6–10], but these efforts have proceeded model by model, and there has been no shared, cumulative library in which machine-checked results about mechanisms accumulate and stay aligned with the models that run.

The Lean proof assistant [11] makes such a library possible. Lean is, at the same time, an efficient functional programming language with an unusually rich type system, and a system for stating and mechanically checking proofs, so *theorems are about the same code that runs*. Lean has been used, increasingly with the help of AI systems, to formalize and check large bodies of mathematics [12]. CogLean applies this model to cognitive science, seeking to align computational models of cognitive mechanisms with assertions about those models and the phenomena they produce. One caution belongs here, at the start. The checker establishes that conclusions follow from premises. It does not establish that the premises are true of people; that remains the business of experiment. What is discovered this way is discovered about the theories, and its psychological significance depends on empirical assumptions that the paper tries to keep in view. What the checker adds is that the entailments, once checked, stay checked, and can be built upon.

This paper is an invitation, not a tutorial. No Lean is needed to read it; where a few lines of Lean appear, they are there to show what a theorem looks like, and each is paraphrased in English beside it. After a short account of what CogLean is (Section 2), the heart of the paper (Section 3) takes the four questions above in turn and answers each with one demonstration from the library. The demonstrations are modest on purpose: a chapter checked, an impossibility proved, a composition that works, a table of theories against laws. Each is meant to show a kind of thing CogLean can do, in a form that a reader could imagine doing with a theory of their own. The Discussion (Section 4) says what the four answers have in common, and the paper ends (Section 5) by saying what different kinds of cognitive scientists could contribute.

One can approach CogLean from many directions. The most obvious, and potentially the most useful, is for researchers working on cognitive architectures to state their architectures' assertions in Lean. It would also be useful for experimental psychologists to state phenomena in CogLean and, where possible, the "mid-level" mechanisms they propose for them. I leave the architectures to the architects, and the phenomena to the psychologists. For historical and personal reasons, the demonstrations here lean toward theories I know from the inside, and toward the composition of mechanisms. I sincerely hope that readers will *not* simply follow my lead, but will invent their own ways of using Lean in cognitive science, and in so doing extend the CogLean repository and methodology.

2 CogLean and CogLib

CogLean is the name I use for the overall project of applying Lean in cognitive science. CogLib is a shared library of Lean objects.[2] CogLib follows the model of Mathlib [11, 12]. CogLib proposes no theory of cognition, and complements cognitive architectures rather than competing with them.

CogLean has four layers. Lean's core is the language and checker; probabilities are exact rationals.[3] CogLib, the general library, holds representations, memories, processes, an interface any memory can implement called a "retriever" (`Cog.Retriever`: study an item; retrieve a distribution over stored items), and theory-neutral phenomena and laws with their theorems. A theory is a package that states its postulates as choices of CogLib constructs, cites its sources, and proves its phenomena and its mechanisms' laws. An application is a task model built from theories and library mechanisms. Nothing in CogLib depends upon any particular theory.

Two features make CogLib useful, not merely rigorous. First, in Lean a definition is both a program and a mathematical object, so a model's theorems are about the definitions that run; when a mechanism changes, every explanation depending on it is rechecked. Second, because mechanisms, phenomena and laws are shared, a result proved once is available to every theory using the same constructs, and can be proved for a whole class of mechanisms satisfying stated laws.

A few terms recur throughout, and a glossary (Appendix B) collects them. A *mechanism* is a definition of a process, such as how a net sorts an encoded stimulus. A *phenomenon* is a predicate over a task's outcomes that names no mechanism; it asserts nothing until some mechanism is proved to satisfy it. A *law* is a property of a mechanism, such as that a studied item retrieves itself for certain; in a theorem it is a hypothesis, not a Lean `axiom`, just as the group axioms are in Mathlib. A theorem's *conditions* are its other hypotheses: situational, unexplained empirical premises, or matters of well-formedness. Every theorem in this paper is fully proved, with no `sorry` (Lean's placeholder for a missing proof), and every Lean name is given in full so that the reader can find it in the repository.

3 Four demonstrations

3.1 Does a theory entail what its authors said? The 1984 chapter

In 1984, Bob Siegler and I proposed a model of how young children choose between retrieving an arithmetic answer and working it out on their fingers [13]. The model's central claim was that errors, solution times and overt strategy use "are all functions of a single variable: the distribution of associations between problems and potential answers" (p. 238). A child has, for each problem, a set of associations of varying strength to possible answers. On a trial the child sets a *confidence criterion* and a *search length*, samples an answer in proportion to its strength, states it if its strength clears the criterion, and tries again up to the search length; if nothing clears the criterion, the child falls back on fingers, and then on counting. Stated answers are strengthened. The chapter argued its predictions in prose and supported them with a simulation. Forty years later, I wanted to know which of them actually follow from the postulates.

CogLean's version of the model (`StrategyChoice/`) states the six postulates of the chapter as choices from CogLib, each with its page citation, and then proves its phenomena from them. Some of the chapter's claims come out exactly as written. Retrieval is used with probability $1-(1-R)^n$, where R is the chance that a single search clears the criterion and n the search length

[2] There should probably eventually be a division between "core" and other sorts of objects.

[3] It is possible to develop content that uses dense and infinite arithmetic by incorporating Mathlib, but I've chosen not to do that here, as it is not needed.

(`SC.retrieval_prob`). The share of errors among retrieval trials is the wrong answers' share of the strength above the criterion, whatever the search length (`SC.retrieval_error_share`). And the chapter's self-described "nonintuitive prediction" (p. 245), that the error rate on counting trials does not depend on the associations at all, is a theorem:

```
theorem counting_error_share ... (p : Problem) (hT : total as s ≠ 0) :
    prob (trial as s bk cc n p) (countingWrong p) =
      prob (trial as s bk cc n p) isCounting *
        prob (count bk.qs bk.qd p.sum) (fun a => a != p.sum)
```

In words: on any problem, the probability of a wrong answer by counting is the probability of counting at all, times the counting process's own error rate on the correct sum. The associations, the criterion and the search length appear on neither side of that product (`SC.counting_error_share`).

One claim did not survive as written. The chapter says that "a peaked distribution results in less frequent use of overt strategies" (p. 245), which is intuitive: the more strength sits on the correct answer, the more often retrieval should succeed. It is also, in general, false. Moving strength from a wrong answer to the correct one can *raise* overt strategy use. The library's witness is small: for $1+1$, with answers 2, 3 and 1 at strengths .38, .31 and .31, moving one hundredth from the answer 3 to the correct answer 2 gives .39, .30, .31, which is more peaked by any reasonable standard; yet the predicted overt strategy use, averaged over the chapter's twelve confidence criteria, rises from 7/12 to 73/120 (`SC.peaking_can_raise_overt_use`). The reason is that the wrong answer has dropped to the criterion .30 and so no longer clears it, while the correct answer, at .39, was never going to clear the higher criteria anyway. What *is* true is more specific: moving strength to the correct answer never raises the share of errors among retrieval trials, at any criterion (`SC.peaking_lowers_retrieval_errors`), and strengthening a correct answer that already clears the criterion never raises backup use (`SC.correct_strengthening_never_raises_backup`). The chapter's claim needs a condition it does not state.

I find this a useful kind of result, and not only because the chapter is mine. It is a small discovery about a theory I thought I knew: the condition was not in the chapter, and I did not know it was missing until the proof would not go through without it. Nothing here says the model is wrong about children. It says that one of the things its authors believed followed from it does not follow without a further assumption, and it says exactly what the assumption buys. A reader who disagrees can point at the postulate they think I have misread; the postulates are in `Postulates.lean` with page numbers beside them.

3.2 Can a theory be shown unable to produce a phenomenon? Collins and Quillian

Collins and Quillian's hierarchical network [14] is one of the best-known models in cognitive psychology, and one of the best-known arguments against it is one of the best-known phenomena. In the network, concepts are nodes with pointers to their supersets and their properties, a property is stored at the highest node it applies to, and verifying "a canary is an animal" means climbing from *canary* to *bird* to *animal*, each step taking time. Hence the *distance effect*: "a canary is a bird" is verified faster than "a canary is an animal." The trouble, pointed out by Rips, Shoben and Smith [15, 16], is that "a robin is a bird" is verified faster than "a chicken is a bird," though both are one step from *bird*, and that "a chicken is an animal" can be verified faster than "a chicken is a bird," which reverses the distance effect outright. The feature comparison model of Smith, Shoben and Rips explains both: a term's meaning is a set of features, some of them defining, and a statement is verified in one stage if the overall similarity of instance and category is high enough, and only otherwise in a second stage that checks the defining features.

That is the textbook story, and the usual conclusion is that the hierarchical model "cannot" produce typicality effects. CogLean lets us say precisely what that means and prove it. CogLib states three phenomena of verification once, for any verifier at all (`CogLib/CogLib/Phenomena/Verification.lean`). A verifier answers "x is a c" with a truth value and an ordinal cost, and the phenomena are judged against a hierarchy that says which categories are nearer to which. *Distance effect*: of two true statements about one instance, the farther category costs more. *Typicality effect*: two instances equally far below a category, both verified true, at different costs. *Reversal*: an instance verified faster as a member of a farther category than of a nearer one. A reversal contradicts the distance effect (`Cog.not_distanceEffect_of_reversal`).

Collins and Quillian's model is then one verifier among the possible ones, and three theorems say what it can and cannot do, for every network, however large, and every bound on search:

```
theorem cq_distanceEffect (G : SemNet N L) (fuel : Nat) :
    DistanceEffect G fuel (cq G fuel)
theorem cq_no_typicality  (G : SemNet N L) (fuel : Nat) :
    ¬ TypicalityEffect G fuel (cq G fuel)
theorem cq_no_reversal    (G : SemNet N L) (fuel : Nat) :
    ¬ Reversal G fuel (cq G fuel)
```

The first says the model always shows the distance effect; the second and third say it *never* shows a typicality effect or a reversal (`SemanticMemory.cq_distanceEffect`, `SemanticMemory.cq_no_typicality`, `SemanticMemory.cq_no_reversal`). The proofs are short, because the reason is simple: in this model cost is a function of distance and nothing else, so two instances at the same distance cost the same. What the theorems add to the textbook argument is their scope. They are not about Collins and Quillian's Figure 1, though the library reproduces it and its six sentence types in the paper's order (`SemanticMemory.Fig1.sentence_types`); they are about every network the model could be given. No choice of hierarchy, properties or exceptions can rescue it, because the claim is about the mechanism.

The feature comparison model is a second verifier held to the same three phenomena. With features for *robin*, *chicken*, *bird* and *animal*, and the two stage-one criteria at 1/4 and 3/4, a robin is a bird in one stage and a chicken in two (`SemanticMemory.Birds.typicality`), and a chicken is an animal in one stage but a bird in two (`SemanticMemory.Birds.reversal`), so the model lacks the distance effect at that setting (`SemanticMemory.Birds.no_distanceEffect`). These are results at one setting, witnesses that the model *can* do what the network model never can; the general companion is that an instance at least as similar to the category as one verified true is never slower (`SemanticMemory.fc_typicality`). The formalization makes its simplifications as declared choices: similarity is the share of the category's features the instance has, fixed rather than sampled, and cost counts stages. Someone who thinks those choices matter can change them and rerun the proofs.

The pattern of this demonstration is the one I most want to convey. Phenomena were stated once, apart from any mechanism. Two rival theories were each written as a mechanism and held to the same phenomena. One was shown to produce a phenomenon always and another never; the other was shown to produce it at a setting. The "never" is the interesting part, because it is the kind of claim that simulation cannot make and that argument makes only informally.

3.3 Do theories compose, and can an explanation be shared or rivalled? Multi-digit addition

The third demonstration is the one that forced most of CogLib into general form, and it is a little longer than the others because it carries three results at once: that independently formalized

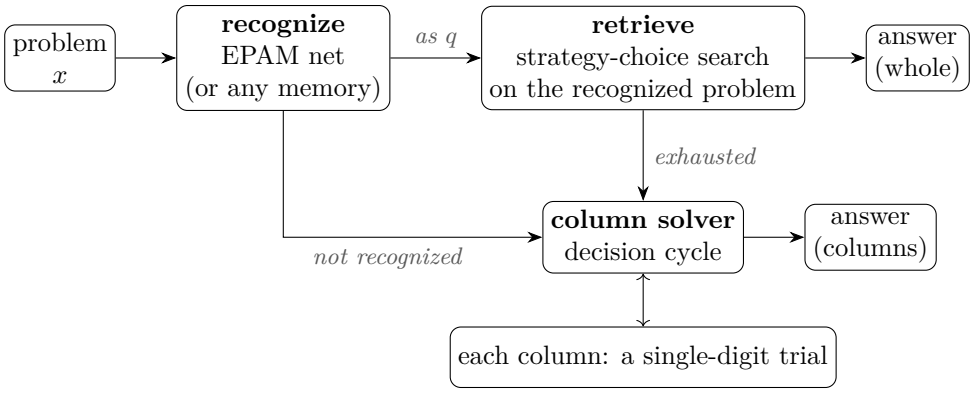


Figure 1: One trial of the integrated model (`MD.Integrated.itrial`).

theories compose into a working model; that an explanation proved for the composition can be lifted to a whole class of memories; and that a rival theory can reach the same phenomenon by a different route, with an experiment that would tell the two apart.

The composition. A child adding two three-digit numbers may know the answer to a familiar problem such as $100+100$, or may work it out column by column. The model (Figure 1) codes each problem by features of its digits, recognizes it with an EPAM discrimination net [17, 18] of familiar problems, and retrieves a whole answer from the recognized problem's associations by the strategy-choice model's sampling (Section 3.1). If nothing is recognized, or retrieval fails, a column solver runs on CogLib's decision cycle, with impasses and repairs after Brown and VanLehn [19], and each column's addition is itself a single-digit strategy-choice trial. Nothing is handed the answer: 250 for $150 + 100$ is assembled from three single-digit trials and a carry rule. The three theories were formalized separately, and none was designed with the others in mind. What joins them is CogLib's general composition "recognize, then retrieve, else back up" (`Cog.recognizeRetrieve`), with the solver as backup. The solver is a problem-solving procedure, and a theorem (`MD.Solver.bridge`) proves that it gives exactly the same answer distributions as a direct column-by-column calculation, so whatever is proved about the simpler description holds of the running procedure.

A phenomenon, stated without a mechanism. In the strategy-choice model alone, a problem's errors and its backup use come from one distribution, so errors and hesitation go together: a child who is often wrong on a problem is also a child who often resorts to fingers on it. Recognition can break that tie. If an unfamiliar problem x is mistaken for a familiar one q, the child retrieves q's answer, confidently and wrongly. CogLib states the resulting pattern once, as a predicate over distributions of answers and the routes that produced them (`Cog.Phenomena.ConfusionErrorPattern`):

```
def ConfusionErrorPattern trial retrievedWrong isRetrieved isBackup x q : Prop :=
  prob (trial q) (retrievedWrong q) = 0 ∧                              -- (i)
  prob (trial x) (retrievedWrong x) = prob (trial x) isRetrieved ∧     -- (ii)
  prob (trial x) isBackup = prob (trial q) isBackup                    -- (iii)
```

(i) The familiar q is never answered wrongly by retrieval; (ii) every retrieved answer to x is wrong; (iii) x backs up exactly as often as q. No mechanism is named. The pattern holds for the composition, for every encoding, net, set of strengths, criterion, search length and backup, whenever q is recognized as itself, x is recognized as q, q's associations are *sharp* (only its correct answer clears the criterion), and the two correct answers differ (`Cog.RR.confident_errors`; for the integrated

model, `MD.Integrated.integrated_confident_errors`). At the illustrative setting, a net that has learned $100 + 100$ and $100 + 200$ confuses $150 + 100$ with $100 + 100$ (`MD.Example.net_facts`) and answers it 200 by retrieval with probability 45/46 (`MD.Example.example_values`). Sharpness is an empirical premise the library does not yet derive; the theorem says what follows from it, not where it comes from.

The same explanation for every memory of a kind. The theorem just described is about EPAM's net. But what it uses of the net is only three properties, and CogLib states them as laws against the retriever interface, so that any memory can be asked whether it has them: *determinacy* (retrieval calls up a single item for certain), *certainty* (a studied item retrieves itself) and *stability* (studying one item leaves another's self-retrieval unchanged). `Cog.RR.confident_errors_of_laws` proves the pattern for *any* memory with the three laws. EPAM's net has all three (`EPAM.determinate`, `EPAM.certainty`, `EPAM.stability`). A new theory of memory inherits the explanation by proving the laws, without touching the proof. The laws are sufficient, not shown necessary, and no claim is made that human memory satisfies them.

A rival reaches the pattern another way. ACT-R's declarative memory, as formalized here (base-level activation, partial matching and a retrieval threshold, at decay 1 and temperature 1; 20)[4] has none of the three laws at any setting with a positive failure weight θ, starting from an empty memory (`ACTR.never_determinate`, `ACTR.never_certainty`, `ACTR.never_stability`; the stability result also needs a nonnegative mismatch penalty and two distinct stimuli). So the class theorem does not cover it, and at the illustrative setting the exact pattern indeed fails (`MD.Integrated.setting_actr_not_pattern`). Not covered is not the same as cannot. A graded version of the pattern needs no certainty: if x calls up q, and q calls up itself, each with probability at least $1 - \delta$, the pattern holds within 2δ (`Cog.RR.confident_errors_graded`). The ACT-R memory meets that condition when q is *primed*: for every $\delta > 0$, enough priming suffices (`Cog.RR.act_priming_confident_errors`), and having just seen q is such priming, through ACT-R's own base-level activation (`Cog.RR.act_recency_confident_errors`). Confident errors therefore have two explanations in the library, from different mechanisms: one from a memory that recognizes by discrimination, one from a memory that is pushed toward a recently seen item.

The observation that would tell them apart. Two explanations of the same pattern invite the question of where they differ, and here they differ on *familiar* problems. EPAM's confusions follow the features a problem shares with a stored one, and a familiar problem is always recognized as itself, so the familiar $100 + 200$ is never answered wrongly (`MD.ExampleACTR.epam_q2_never_wrong`). Priming, by contrast, draws retrieval toward the primed item from *every* other item: just after seeing $100 + 100$ ten times, the formalized ACT-R memory answers the stranger $150 + 100$ with 200 with probability 4725/5129, and the familiar $100 + 200$ with 200 with probability 4725/6854 (`MD.ExampleACTR.recency_values`). The two explanations thus make opposite predictions about what priming a familiar problem does to a child's *other* familiar problems: nothing, on one account; errors, on the other. As far as I know the experiment has not been run. It is not hard to describe: prime one familiar problem by repetition, then measure errors on other familiar problems against

[4]This is a simplified ACT-R declarative memory, and its fidelity to ACT-R is partial. With a single chunk of weight w, the retrieval probability $w/(\theta + w)$ has the logistic form ACT-R uses. With several chunks, however, retrieval failure here competes with all the chunks inside one softmax, whereas ACT-R selects the most active chunk under noise and then compares its activation with the threshold. The mismatch penalty is also a multiplicative discount in the code, μ^m for m mismatching features, not ACT-R's additive penalty on activation. Its values therefore do not map onto the usual ACT-R parameter settings.

errors on strangers. It is the kind of experimental question that formal comparison can produce, and it came out of the library rather than being put in.

Two caveats. First, the pattern is stated over internal routes, retrieval versus backup, not over observed responses. Applying it to children needs a measurement interpretation of route, for instance by latency or self-report, and infrequent backup is not the same thing as reported confidence; each is a further bridge (Appendix A). Second, the model is a demonstration of composition, not a theory of arithmetic: carrying is error-free, the columns are independent, and the strengths at the illustrative setting are chosen, not fitted. The earlier edition of this introduction [21] carries the multi-digit model in full, with its further consequences and its exact values.

3.4 Can theories be compared by what they entail? The memory theory matrix

The class theorem of the previous section needed an interface that any memory can implement and laws stated against that interface. Once those exist, a different kind of comparison becomes possible, and it is the fourth demonstration. Theories of memory are usually compared by fitting each to data, one experiment and one parameter setting at a time, and their qualitative commitments (whether studying one item can disturb another, whether the order of study matters, whether anything unstudied can be retrieved) are left implicit in their implementations. With laws stated once, any memory theory can be asked whether it has a law, and the answer can often be proved for a whole range of parameter settings at once. Laid out as a table of theories by laws, the answers form a *theory matrix*: a classification of theories by what they entail, not by how well they fit.

Six laws are stated once, against the retriever interface (`CogLib/CogLib/Phenomena/RetrievalLaws.lean`): certainty, stability and determinacy, as above; *order-freedom* (swapping the last two study events leaves retrieval unchanged); *recency* (of two stimuli studied last and equally similar to the probe, the later is retrieved more often); and *stored-only* (nothing unstudied is retrieved). Here is the first of them, to show what a law looks like:

```
def Certainty (R : Retriever S Item) (s0 : S) (U : List Item) : Prop :=
  ∀ s, Reachable R s0 U s → ∀ y ∈ U, pRet R (R.study s y) y y = 1
```

In words: in any memory state reachable by studying items from the universe U, studying an item y and then probing with y retrieves y with probability one. The laws are not independent: an order-free memory lacks recency wherever recency can be tested (`Cog.Laws.not_recency_of_orderFree`).

Four *formalized families* are classified (Table 1): the formalized EPAM family (discrimination nets), the formalized ACT-R family (declarative memory, simplified as in the footnote above), the formalized strategy-choice family (a strength memory with sampling and a criterion) and the formalized global-matching family (after MINERVA 2; 22). Each family is a narrow formalization of one regime of its theory, with the parameters its definitions expose, and not the theory as its authors hold it; a verdict about a family is not a verdict about the theory. Each family proves its own row. A verdict is read from its theorem's statement. If the theorem binds the family's parameters, it holds at every setting: the family **has** the law, or **never** has it. Otherwise it is a counterexample at one setting (*lacks*). A generator builds the table from the theorems, checks that names agree with statements, and checks each cell's axioms. Every law holds for some family and fails for another, and no two families share a row.

Formalized family	Certainty	Stability	Order-free	Recency	Stored only	Determinate
EPAM	**has**	**has**	lacks	lacks	**has**	**has**
ACT-R	**never**	**never**	**never**	**has**	**has**	**never**
Strategy choice	lacks	lacks	**has**	**never**	**has**	lacks
Global matching	lacks	lacks	**has**	**never**	lacks	**has**
Adopted interpretation of the evidence	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

Table 1: The memory theory matrix. Each row is a formalized family, a narrow formalization of one regime of the named theory, not the theory itself. Above the line, each verdict is a theorem; **has** and **never** hold across the range of parameters and stimuli each theorem states (for ACT-R, a positive failure weight θ, a nonnegative mismatch penalty μ, positive for recency, an empty starting memory, and two distinct stimuli where a law compares them), and lacks is a counterexample at one setting. The last row is not a theorem but an illustrative interpretation of the evidence about people, with sources and bridges (`notes/evidence.json`). The theorems are in `EPAM/EPAM/Laws.lean`, `ACTR/ACTR/Laws.lean`, `StrategyChoice/StrategyChoice/Laws.lean` and `GlobalMatching/GlobalMatching/Laws.lean`.

The last row records, outside Lean, one interpretation of what experiments show about people. It is illustrative: it shows how evidence attaches to laws, not which theory is right. Each judgment rests on a bridge from what an experiment measures (reports, decisions) to what a law concerns (what memory retrieves), and the bridges are not formalized (Appendix A). Some are weak. Certainty concerns probing with a stimulus just after studying it; that cued recall retrieves words free recall missed [23] bears on it only indirectly. Recency is a strict comparison between the last two, equally similar stimuli; serial-position effects [24, 25] are averages over lists. Stability [26–28], stored-only retrieval [29, 30] and determinacy are recorded as mixed. A family would be incompatible with the interpretation of a law when its verdict holds at every setting and the judgment goes the other way.

The formal verdicts are the result: four formalized families classified against six laws, each verdict proved. Reading them against people is a further step, only as good as the bridges, and no conclusion about which theory of memory is right is drawn here. What the matrix does is make such questions precise and extensible: a new memory adds a row, a new law a column, a better bridge can replace a weak one, and disagreeing with a judgment is a precise act with traceable consequences.

4 Discussion

The four demonstrations answer four different questions, but they share one shape, and the shape is the point of CogLean. In each, something was stated once, apart from any particular theory: a phenomenon of verification, a pattern of confident errors, a law of retrieval. Then one or more theories, each written as a mechanism, were held to it, and what each entails was proved rather than argued or sampled. The strategy-choice chapter was held to its own claims; two models of semantic memory were held to three phenomena; three theories were composed and the composition held to a pattern that none of them produces alone, after which a fourth theory was held to the same pattern; and four memories were held to six laws. Two of the four did what the introduction promised first: they wrote down what we already believed, precisely, and established its exact scope. That Collins and Quillian's network cannot show typicality was folklore; now it is a theorem about every network. The other two went further and found things that were not known: a condition missing from a

published claim, and a prediction on which two explanations of the same pattern disagree. Neither result is about children directly. Each is about a theory, and what it means for children depends on the bridges of Appendix A. That distinction is not a weakness. It is what lets the results be stated exactly and contested precisely. Because the phenomena and laws are shared, each result is available to the next theory that uses the same constructs, and the explanations themselves become reusable objects. A theorem characterizes a class of mechanisms; another theory joins the class by proving its laws, or reaches the same phenomenon by a different route; and the resulting explanations can be compared, composed and challenged.

CogLean's targets are *phenomena*, regularities of behavior stated without mechanisms; observed, a phenomenon is a psychological fact, and derived but unobserved, a prediction. Its central theorems are meant as *explanations*: deductions of a phenomenon from mechanisms and stated assumptions [31], carried through the operation of mechanisms [32]. Not every valid proof explains [33]; which theorems explain is a judgment the library records openly. A phenomenon stated over internal routes, like the confusion-error pattern, is a computational property until a measurement interpretation connects those routes to behavior (Appendix A).

Proof, experiment and simulation can each expose the limits of the others. Lean checks that conclusions follow from premises. Whether premises hold of people is for experiment, and whether a theorem's conditions arise when a model runs is for simulation. An independent simulation of the multi-digit model found valid theorems whose hypotheses were never realized in its learning histories [34]. Every theorem in CogLean is *valid*: Lean has checked that its conclusion follows from its premises. That guarantee is the foundation of everything else, but it is not enough. Validity is neither empirical adequacy, whether the premises hold of people, nor explanation. A theorem whose premises simply restate its phenomenon is valid but empty, so the repository records, for each headline result, how far it lies from its premises. What separates explanation from restatement is often a second consequence of the same mechanism, as when the EPAM and ACT-R explanations of confident errors differ on familiar problems.

CogLean is not the first formalization of cognitive theory. Soar was specified in Z [6]; Sceptic is an executable specification language for cognitive models [7]; ACT-R was reconstructed to separate its commitments from its implementation [8], given an operational semantics with a provably correct translation [9, 35], and translated to timed automata for exhaustive analysis [10, 36]; a common formal ground for architectures has been proposed [37]; Soar models have been compiled for probabilistic model checking [38, 39]; verbal theories have been formalized as logic programs and automata [40, 41]; and critical tests of ACT-R have been derived from its core assumptions [42]. Each of these brought formal rigor to one theory, and several went much further than CogLean in their own direction. CogLean differs in three ways. The model and the formal object are the same thing: the definition that runs is the definition the theorems are about. Its theorems hold for classes rather than instances: a model checker analyzes one specified model exhaustively, even with symbolic parameters [43], whereas a CogLean theorem covers every mechanism satisfying stated laws. And it works across theories rather than within one, and is meant to be cumulative, a library that others extend, as Mathlib is, rather than a formalization that lives in a single paper. These are differences of kind, not of scale, and they come at a cost: model checking can verify large, detailed models that would be laborious to reason about deductively, Gall's semantics of ACT-R is far more faithful to ACT-R than the simplified memory used here, and CogLean's models are small. A model checker can tell us what a particular model does; a theorem tells us what every model of a kind must do.

The framework does not establish psychological plausibility. Key conditions, such as sharp associations, are assumed, not derived, and the examples' parameters are chosen, not learned. Exact rationals restrict some theories to special cases: the ACT-R memory appears at decay 1

and temperature 1. Exact finite distributions will not scale to continuous or neural models without other mathematics. And the bridges from what the theorems concern to what experiments measure are stated, not formalized.

5 An invitation

Newell asked how cognitive science could become cumulative. CogLean is not a theory of cognition, and it is not meant to be a collection of independently formalized models. It is meant to become what Mathlib is for mathematics: a cumulative library of mechanisms, phenomena, laws and explanations, in which each contribution enables further deductions, and in which complex models stay aligned with equally complex explanations of their behavior as the library grows and changes. In such a library researchers with different commitments can state, compare, criticize and improve explanations, and disagreements about assumptions, mechanisms, evidence and predictions become explicit objects of study. Cognitive science can then accumulate not only experiments and models, but also what can be established about the relationships among theories and phenomena, and can reason its way to things it did not previously know.

The demonstrations suggest what different kinds of cognitive scientists could contribute, and most of it needs no Lean. Experimentalists could contribute phenomena: a regularity stated precisely enough to become a target, like the three phenomena of verification, is a contribution in itself, and so is a judgment about the evidence, like the last row of the matrix, or a test of a prediction the library has produced, like the priming experiment of Section 3.3. Modelers could contribute mechanisms, and could challenge the representations the existing ones assume; a theory enters the library by stating its postulates as choices of CogLib constructs, as the strategy-choice model does, and a reader who thinks a postulate has been misread can say which. Mathematical psychologists could contribute laws and class theorems: the matrix has four rows and six columns, and both are open. Methodologists could contribute bridges, the assumptions that connect what a theorem concerns to what an experiment measures, which this paper leaves informal and which are themselves pieces of psychological theory. And researchers in formal methods could contribute abstractions, proofs and tooling, including ways to make all of this easier for everyone else. As AI systems get better at proofs, separating the social work of proposing, contesting and interpreting theories from the mechanical checking of their consequences may matter more, and much of the Lean in this repository was in fact written that way.

This paper has followed one path through CogLean. It is only one of many, and I hope that others will take CogLean in directions of their own.

6 Availability

Everything in this paper is open source and available in CogLean-Intro [44], a small, self-contained edition of the library; the earlier edition [21] carries the multi-digit model in full, and the CogLean repository [34] holds the wider library. Every theorem is fully proved, with no `sorry`, and depends only on Lean's standard axioms.

7 Acknowledgements

The CogLean concept grew out of conversations between Chris Bates, Pete Pirolli, and the author. Bates and Pirolli, as well as John Laird and Christian Lebiere provided useful early guidance. Almost all of the Lean programming in the CogLean project was accomplished by Claude Code

(mostly Opus 5.5) with extensive guidance from the author, as well as ongoing feedback from ChatGPT as well as various cognitive scientists including those named above. Papers and presentations were initially drafted by LLMs and then substantially rewritten by the author. Any remaining errors, including hallucinations, are the author's alone.

References

[1] Allen Newell. You can't play 20 questions with nature and win: Projective comments on the papers of this symposium. In William G. Chase, editor, *Visual Information Processing*, pages 283–308. Academic Press, New York, 1973.

[2] John E. Laird, Allen Newell, and Paul S. Rosenbloom. SOAR: An architecture for general intelligence. *Artificial Intelligence*, 33(1):1–64, 1987. doi: 10.1016/0004-3702(87)90050-6.

[3] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, Cambridge, MA, 2012.

[4] John R. Anderson. *How Can the Human Mind Occur in the Physical Universe?* Oxford University Press, New York, 2007.

[5] John E. Laird, Christian Lebiere, and Paul S. Rosenbloom. A standard model of the mind: Toward a common computational framework across artificial intelligence, cognitive science, neuroscience, and robotics. *AI Magazine*, 38(4):13–26, 2017. doi: 10.1609/aimag.v38i4.2744.

[6] Brian G. Milnes, Garrett Pelton, Robert Doorenbos, Mike Hucka, John Laird, Paul Rosenbloom, and Allen Newell. A specification of the Soar cognitive architecture in Z. Technical Report CMU-CS-92-169, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, 1992.

[7] Richard Cooper, John Fox, Jonathan Farringdon, and Tim Shallice. A systematic methodology for cognitive modelling. *Artificial Intelligence*, 85(1–2):3–44, 1996. doi: 10.1016/0004-3702(95)00112-3.

[8] Terrence C. Stewart and Robert L. West. Deconstructing and reconstructing ACT-R: Exploring the architectural space. *Cognitive Systems Research*, 8(3):227–236, 2007. doi: 10.1016/j.cogsys.2007.06.006.

[9] Daniel Gall and Thom Frühwirth. An operational semantics for the cognitive architecture ACT-R and its translation to Constraint Handling Rules. *ACM Transactions on Computational Logic*, 19(3):22:1–22:42, 2018. doi: 10.1145/3218818. Preprint arXiv:1702.01606 (2017).

[10] Vincent Langenfeld, Bernd Westphal, and Andreas Podelski. On formal verification of ACT-R architectures and models. In *Proceedings of the 41st Annual Conference of the Cognitive Science Society*, pages 618–624, 2019.

[11] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

[12] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

[13] Robert S. Siegler and Jeff Shrager. Strategy choices in addition and subtraction: How do children know what to do? In Catherine Sophian, editor, *Origins of Cognitive Skills*, pages 229–293. Erlbaum, Hillsdale, NJ, 1984.

[14] Allan M. Collins and M. Ross Quillian. Retrieval time from semantic memory. *Journal of Verbal Learning and Verbal Behavior*, 8(2):240–247, 1969.

[15] Lance J. Rips, Edward J. Shoben, and Edward E. Smith. Semantic distance and the verification of semantic relations. *Journal of Verbal Learning and Verbal Behavior*, 12(1):1–20, 1973.

[16] Edward E. Smith, Edward J. Shoben, and Lance J. Rips. Structure and process in semantic memory: A featural model for semantic decisions. *Psychological Review*, 81(3):214–241, 1974.

[17] Edward A. Feigenbaum. An information processing theory of verbal learning. RAND Paper P-1817, RAND Corporation, Santa Monica, CA, 1959.

[18] Edward A. Feigenbaum and Herbert A. Simon. A theory of the serial position effect. *British Journal of Psychology*, 53(3):307–320, 1962. doi: 10.1111/j.2044-8295.1962.tb00836.x.

[19] John Seely Brown and Kurt VanLehn. Repair theory: A generative theory of bugs in procedural skills. *Cognitive Science*, 4(4):379–426, 1980. doi: 10.1207/s15516709cog0404_3.

[20] John R. Anderson and Christian Lebiere. *The Atomic Components of Thought*. Erlbaum, Mahwah, NJ, 1998.

[21] Jeff Shrager. CogLean-20261106-Intro: A small introduction to CogLean. `https://github.com/jeffshrager/CogLean-20261106-Intro`, 2026.

[22] Douglas L. Hintzman. MINERVA 2: A simulation model of human memory. *Behavior Research Methods, Instruments, & Computers*, 16(2):96–101, 1984.

[23] Endel Tulving and Zena Pearlstone. Availability versus accessibility of information in memory for words. *Journal of Verbal Learning and Verbal Behavior*, 5:381–391, 1966.

[24] Bennet B. Murdock. The serial position effect of free recall. *Journal of Experimental Psychology*, 64:482–488, 1962.

[25] Murray Glanzer and Anita R. Cunitz. Two storage mechanisms in free recall. *Journal of Verbal Learning and Verbal Behavior*, 5:351–360, 1966.

[26] John A. McGeoch. Forgetting and the law of disuse. *Psychological Review*, 39:352–370, 1932.

[27] Jean M. Barnes and Benton J. Underwood. "Fate" of first-list associations in transfer theory. *Journal of Experimental Psychology*, 58:97–105, 1959.

[28] Roger Ratcliff, Steven E. Clark, and Richard M. Shiffrin. List-strength effect: I. data and discussion. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 16:163–178, 1990.

[29] Michael I. Posner and Steven W. Keele. On the genesis of abstract ideas. *Journal of Experimental Psychology*, 77:353–363, 1968.

[30] Henry L. Roediger and Kathleen B. McDermott. Creating false memories: Remembering words not presented in lists. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 21:803–814, 1995.

[31] Carl G. Hempel and Paul Oppenheim. Studies in the logic of explanation. *Philosophy of Science*, 15(2):135–175, 1948.

[32] Peter Machamer, Lindley Darden, and Carl F. Craver. Thinking about mechanisms. *Philosophy of Science*, 67(1):1–25, 2000.

[33] Mark Steiner. Mathematical explanation. *Philosophical Studies*, 34(2):135–151, 1978.

[34] Jeff Shrager. CogLean: A library of cognitive theory in Lean 4. `https://github.com/jeffshrager/CogLean`, 2026.

[35] Daniel Gall and Thom Frühwirth. A decidable confluence test for cognitive models in ACT-R, 2017. arXiv:1705.08392.

[36] Vincent Langenfeld, Bernd Westphal, and Andreas Podelski. A formal operational model of ACT-R: Structure and behaviour. In *Proceedings of the 43rd Annual Conference of the Cognitive Science Society*, 2021.

[37] Marco Ragni, Kai Sauerwald, Tanja Bock, Gabriele Kern-Isberner, Paulina Friemann, and Christoph Beierle. Towards a formal foundation of cognitive architectures. In Chuck Kalish, Martina A. Rau, Xiaojin Zhu, and Timothy T. Rogers, editors, *Proceedings of the 40th Annual Conference of the Cognitive Science Society*, Madison, WI, 2018.

[38] Parth Ganeriwala, Candice Chambers, James I. Lathrop, Siddhartha Bhattacharyya, Junaid Babar, Stephen B. Gilbert, Isaac Amundson, Michael C. Dorneich, and Mohammed Abdul Hafeez Khan. Formal analysis of stochastic cognitive models: A translation framework from Soar to PRISM. In *NASA Formal Methods*, Lecture Notes in Computer Science, pages 192–213, Cham, 2026. Springer. doi: 10.1007/978-3-032-28079-4_9.

[39] Marta Kwiatkowska, Gethin Norman, and David Parker. PRISM 4.0: Verification of probabilistic real-time systems. In *Computer Aided Verification (CAV 2011)*, volume 6806 of *Lecture Notes in Computer Science*, pages 585–591. Springer, 2011. doi: 10.1007/978-3-642-22110-1_47.

[40] Jean-Christophe Rohner and Håkan Kjellerstrand. Using logic programming for theory representation and scientific inference. *New Ideas in Psychology*, 61:100838, 2021. doi: 10.1016/j.newideapsych.2020.100838.

[41] Alain Finkel, Gaspard Fougea, and Stéphane Le Roux. An automata-based method to formalize psychological theories: The case study of Lazarus and Folkman's stress theory. In *Proceedings of MODELSWARD 2025*, 2025. arXiv:2501.05185.

[42] Christopher R. Fisher, Joseph W. Houpt, Othalia Larue, and Kevin Schmidt. Using systems factorial technology for global model analysis of ACT-R's core architectural assumptions. *Journal of Mathematical Psychology*, 125:102924, 2025. doi: 10.1016/j.jmp.2025.102924.

[43] Ernst Moritz Hahn, Holger Hermanns, and Lijun Zhang. Probabilistic reachability for parametric Markov models. *International Journal on Software Tools for Technology Transfer*, 13 (1):3–19, 2011. doi: 10.1007/s10009-010-0146-x.

[44] Jeff Shrager. CogLean-Intro: An invitation to CogLean. `https://github.com/jeffshrager/CogLean-Intro`, 2026.

A From computational properties to psychological phenomena: bridges

A theorem about a model establishes that the model and its assumptions, M, entail a computational property P. Explaining what people do takes two further steps. An observation bridge, B, connects P to a prediction about measurable behavior; an empirical test then compares that prediction with data:

$$\underbrace{M \Rightarrow P}_{\text{computational deduction}} \qquad \underbrace{P + B \Rightarrow O_{\text{predicted}}}_{\text{observation bridge}} \qquad \underbrace{O_{\text{predicted}} \text{ vs. } O_{\text{measured}}}_{\text{empirical test}}$$

Lean checks the first step and can check the second when the bridge is formalized; the third is empirical science. A valid bridge can connect a valid theorem to a prediction that people do not exhibit. The explanation is then empirically inadequate, although every deduction in it is correct.

Two quite different things can be called bridges. An *implementation bridge* relates two formal descriptions; the multi-digit model has one, the equivalence of the column solver and direct column arithmetic (`MD.Solver.bridge`), and Lean proves it from the definitions. An *observation bridge* relates a formal description to measurements. It cannot be proved from definitions, because one side is a claim about people. In the body of the paper, "bridge" means an observation bridge.

The confusion-error pattern (Section 3.3) names no mechanism, but it is stated over internal routes, retrieval and backup. A child's behavior shows an answer, perhaps a response time, perhaps a confidence report, but not the route. Reaching behavior takes assumptions such as these: retrieval is usually fast; backup is usually slower; fast answers given with high confidence usually come from retrieval. With a model of latency and confidence built on them, one could derive an observable prediction: the confused problem yields more fast, confident errors than the familiar one. The prediction is conditional on the bridge, and the bridge may be false. Theory-neutral and observable are different properties of a phenomenon. The verification phenomena of Section 3.2 are closer to observation, since cost is ordinal and the data are response times, but "cost" and "time" are still joined by an assumption.

The matrix's evidence row works the same way. Certainty is a precisely defined law: just after an item is studied, probing memory with that item retrieves it for certain. Cued recall recovers words that free recall missed [23]. Does that contradict certainty? Not necessarily, since neither free nor cued recall is the operation the law describes. Counting the experiment as evidence against the law needs a bridge from the experimental conditions to that operation, and the one recorded in `notes/evidence.json` is weak. None of this weakens the theorems. It locates the further scientific work their empirical reading needs.

An observation bridge is itself a piece of psychological theory, not a translation of terminology: that fast responses reflect retrieval is a substantive hypothesis about cognition. Bridges can therefore be proposed, criticized, compared and tested, and they could be formalized: an observation model, conceptually `observe : Internal → Dist Observable`, would map each internal outcome to a distribution over answers, times and reports, and Lean could then check that a mechanism and an observation model together entail an observable phenomenon. It could not check that the observation model describes people. CogLean does not close the gap between computational models and psychological observations. It makes the gap explicit, lets proposed bridges be stated precisely, and lets their deductive consequences be checked.

B Glossary

The terms below are defined where the paper first uses them; this appendix collects them for reference. Table 2 maps CogLean's constructs onto familiar scientific notions.

Scientific notion	In CogLean
Model (the program)	executable definitions: the same ones that run
Mechanism (postulate)	a CogLib process, chosen by a theory and cited to its source
Phenomenon	a predicate over a task's outcomes, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: outside Lean, recorded with sources
Prediction	a theorem's conclusion about an unobserved phenomenon
Assumption	a condition (a theorem's hypothesis) or a law (a property of a mechanism)
Explanation	a theorem and its lemmas: mechanisms and assumptions entail the phenomenon
Deduction	what Lean's kernel checks
Witness	a value computed at one setting: not an explanation

Table 2: Scientific notions and what they are in CogLean.

Lean

Lean. A functional programming language that is also a proof assistant. A definition is both a program that runs and a mathematical object that theorems are about.

Kernel. The small part of Lean that checks every proof. Deduction, in CogLean, is what the kernel checks.

`sorry`. Lean's placeholder for a missing proof. No CogLean theorem uses it.

Standard axioms. `propext`, `Quot.sound` and `Classical.choice`, the only axioms CogLean's theorems depend on. A law is never an axiom; it is a hypothesis.

Fully qualified name. A declaration's full name, which says where to find it: `Cog.` for CogLib; `SC.` (strategy choice), `SemanticMemory.`, `EPAM.`, `ACTR.` and `GM.` (global matching) for the theories; `MD.` for the multi-digit model.

Exact rationals. Probabilities are fractions (Lean's `Rat`), not floating-point numbers, so values at a setting are computed and checked exactly.

Mathlib. Lean's community library of mathematics, the model for CogLib.

CogLean

CogLean. The project of applying Lean in cognitive science.

CogLib. The shared library: representations, memories, processes, the retriever interface, and theory-neutral phenomena and laws with their theorems. Nothing in it depends on any particular theory.

Theory. A package that states its postulates as choices of CogLib constructs, cites its sources, and proves its phenomena and its mechanisms' laws.

Application. A task model built from theories and library mechanisms, such as the multi-digit model.

Mechanism. A definition of a process, such as how a net sorts a code. A theory's postulate is a choice of mechanism.

Distribution. An exact finite probability distribution over outcomes (`Cog.Dist`).

Retriever. The interface any memory can implement (`Cog.Retriever`): study an item; retrieve a distribution over stored items.

Verifier. The interface any model of sentence verification can implement (`Cog.Verifier`): for "x is a c", a truth value and an ordinal cost.

Phenomenon. A predicate over a task's outcomes, with no mechanism in it: a target. Observed, it is a psychological fact; derived but unobserved, a prediction. One stated over internal routes is a computational property until a measurement interpretation connects the routes to behavior.

Law. A property of a mechanism, stated against an interface, such as certainty. It is a hypothesis of the theorems that use it.

Condition. A theorem's hypothesis: situational, an unexplained empirical premise, or well-formedness.

Explanation. A theorem and its lemmas, showing that mechanisms and assumptions entail a phenomenon. Not every valid proof explains.

Class theorem. A theorem for every mechanism that satisfies stated laws, such as `Cog.RR.confident_errors_of_laws`.

Graded. Holding within a tolerance δ rather than exactly.

Setting. A choice of parameter values.

Witness. A value computed at one setting. It shows that hypotheses can be met and that models diverge, but not why; it is not an explanation.

Verdict. A matrix cell, read from its theorem's statement: **has** or **never** (at every setting), or *lacks* (a counterexample at one setting).

Formalized family. A narrow formalization of one regime of a theory, with the parameters its definitions expose; a row of the matrix. It is not the theory itself.

Bridge. The link from what an experiment measures to what a law or phenomenon concerns. A scientific assumption, not formalized here (Appendix A).

Interpretation of the evidence. The matrix's last row: one illustrative, contestable reading of experiments about people, with sources and bridges (`notes/evidence.json`).

Phenomena and laws

Distance effect. Of two true statements about one instance, the one whose category is farther up the hierarchy costs more.

Typicality effect. Two instances equally far below a category, both verified true, at different costs.

Reversal. An instance verified faster as a member of a farther category than of a nearer one.

Confusion-error pattern. A familiar problem q is never answered wrongly by retrieval, every retrieved answer to a problem x is wrong, and x backs up exactly as often as q (`Cog.Phenomena.ConfusionErrorPattern`).

Certainty. Just after a stimulus is studied, probing with it retrieves it for certain.

Stability. Studying one stimulus leaves the chance that another retrieves itself unchanged.

Determinacy. Retrieval calls up a single item for certain.

Order-freedom. Swapping the last two study events leaves retrieval unchanged.

Recency. Of two stimuli studied last and equally similar to the probe, the later is retrieved more often.

Stored-only. Nothing unstudied is retrieved.

The cognitive models

Strategy choice (DOA). Siegler and Shrager's distributions of associations [13]. Each problem is associated with answers, with varying strengths. A search samples an answer in proportion to strength and states it if its strength clears a *confidence criterion*, for up to n searches (the *search length*); otherwise a backup strategy (fingers, then counting) is used. Its memory, sampling by strength with a criterion, is the matrix's strength family.

Sharp. A problem's associations are sharp when only its correct answer clears the criterion.

Hierarchical network. Collins and Quillian's model [14]: concepts in a hierarchy, properties stored at the highest node they apply to, verification by climbing.

Feature comparison. Smith, Shoben and Rips's model [16]: meanings as feature sets, a first stage on overall similarity, a second on defining features.

EPAM. Recognition by a discrimination net [18]. A stimulus's code, a vector of features, is sorted down the net's tests to a stored *image*; when a stimulus is confused with a stored one, learning adds a test on the first feature where they differ.

Column solver. The backup in the multi-digit model: column-by-column addition on CogLib's decision cycle, with impasses and repairs [19], each column a strategy-choice trial.

ACT-R declarative memory. Retrieval by base-level activation (recency and frequency of study), partial matching and a retrieval threshold [20]; formalized here at decay 1 and temperature 1. *Priming* raises an item's activation.

Global matching. A memory after MINERVA 2 [22], in which a probe is compared with every stored trace at once.

Recognize, retrieve, else back up. CogLib's general composition (`Cog.recognizeRetrieve`): recognize the stimulus, retrieve an answer from what it is recognized as, and back up if nothing is recognized or retrieval fails.

Route. Whether an answer came from retrieval or from the backup.

x and q. A problem and the familiar problem it is taken for, as $150 + 100$ and $100 + 100$.